

Documentation: Obstacle Avoidance Robot

1. Introduction

The Obstacle Avoidance Robot is an autonomous vehicle designed to navigate its environment without human intervention. Using an ultrasonic sensor mounted on a servo motor, the robot constantly scans the area ahead. When it detects an obstacle within a predefined distance, it halts, scans its left and right surroundings to find a clear path, and avoid the collision. This project demonstrates the fundamental principles of autonomous navigation, sensor integration, and intelligent decision-making in robotics.

2. Hardware Components Required

- Arduino UNO (Microcontroller)
- L298N Motor Driver Module
- HC-SR04 Ultrasonic Sensor
- Micro Servo Motor (SG90)
- 2 x DC Gear Motors (BO Motors) with Wheels
- 1 x Caster Wheel (Front)
- 12V Li-ion Battery Power Supply
- Robot Chassis
- Jumper Wires

3. Circuit Connections

Ultrasonic Sensor (HC-SR04) to Arduino:

- **VCC** -> 5V
- **GND** -> GND
- **Trig** -> Pin 13
- **Echo** -> Pin 12

Servo Motor to Arduino:

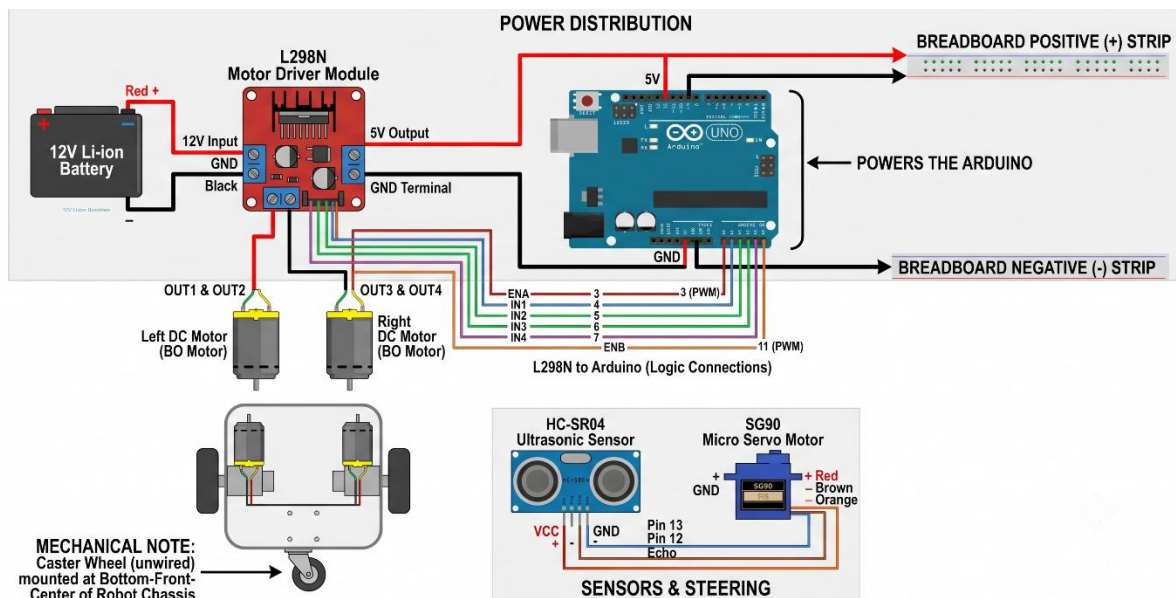
- **Red Wire (Power)** -> 5V
- **Brown/Black Wire (GND)** -> GND
- **Orange/Yellow Wire (Signal)** -> Pin 2

L298N Motor Driver to Arduino:

- **ENA** (Left Motor Speed) -> Pin 3 (PWM)
- **IN1** (Left Motor Forward) -> Pin 4
- **IN2** (Left Motor Backward) -> Pin 5
- **IN3** (Right Motor Forward) -> Pin 6
- **IN4** (Right Motor Backward) -> Pin 7
- **ENB** (Right Motor Speed) -> Pin 11 (PWM)

Power Supply (12V Battery) Connections:

- **Battery (+) Positive** -> L298N (12V input)
- **Battery (-) Negative** -> L298N (GND) **AND** Arduino (GND) -> *Common Ground is mandatory.*
- **L298N (5V Output)** -> Arduino (VIN / 5V pin) -> *To power the Arduino.*



4. Source Code

```
#include <Servo.h>

const int TRIG_PIN = 13;
const int ECHO_PIN = 12;
const int SERVO_PIN = 2;
const int ENA = 3;
const int IN1 = 4;
const int IN2 = 5;
const int IN3 = 6;
const int IN4 = 7;
const int ENB = 11;

const int STOP_DIST = 20;
const int TURN_SPEED = 140;
```

```

int currentSpeed = 0;

Servo scanner;

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  scanner.attach(SERVO_PIN);
  scanner.write(90);
  delay(1000);
}

void loop() {
  scanner.write(90);
  int frontDist = getDistance();

  if (frontDist > STOP_DIST) {
    moveForward();
  } else {
    avoidObstacle();
  }
}

int getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long dur = pulseIn(ECHO_PIN, HIGH, 30000);

  if (dur == 0) {
    return 400;
  }

  return dur * 0.034 / 2;
}

void moveForward() {
  // Adjust RIGHT_MAX or LEFT_MAX if the robot drifts
  int LEFT_MAX = 190;
  int RIGHT_MAX = 150;

  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  while (currentSpeed < LEFT_MAX) {
    currentSpeed += 10;
  }
}

```

```

        if (currentSpeed > LEFT_MAX) {
            currentSpeed = LEFT_MAX;
        }

        int rightSpeed = currentSpeed;
        if (rightSpeed > RIGHT_MAX) {
            rightSpeed = RIGHT_MAX;
        }

        analogWrite(ENA, currentSpeed);
        analogWrite(ENB, rightSpeed);
        delay(15);
    }
}

void moveBackward() {
    analogWrite(ENA, TURN_SPEED);
    analogWrite(ENB, TURN_SPEED);

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

void stopRobot() {
    currentSpeed = 0;

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

void turnLeft() {
    analogWrite(ENA, TURN_SPEED);
    analogWrite(ENB, TURN_SPEED);

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);

    delay(300);
    stopRobot();
    delay(200);
}

void turnRight() {
    analogWrite(ENA, TURN_SPEED);
    analogWrite(ENB, TURN_SPEED);

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);

    delay(300);
    stopRobot();
    delay(200);
}

```

```

}

int lookLeft() {
    scanner.write(170);
    delay(500);
    int dist = getDistance();

    scanner.write(90);
    delay(200);

    return dist;
}

int lookRight() {
    scanner.write(10);
    delay(500);
    int dist = getDistance();

    scanner.write(90);
    delay(200);

    return dist;
}

void avoidObstacle() {
    stopRobot();
    delay(200);

    moveBackward();
    delay(400);

    stopRobot();
    delay(300);

    while (true) {
        if (lookLeft() > STOP_DIST) {
            turnLeft();
            break;
        }

        if (lookRight() > STOP_DIST) {
            turnRight();
            break;
        }

        moveBackward();
        delay(400);

        stopRobot();
        delay(200);
    }
}

```

5. Issues Faced and Troubleshooting

During the development and testing of this robot, several hardware and software challenges were encountered and successfully resolved:

Issue 1: Robot "Jumping" or performing a "Wheelie" on Start

- **Problem:** When applying full speed (PWM 255) from the 12V Li-ion battery, the sudden torque caused the front of the robot (caster wheel) to lift off the ground, causing a jumping effect.
- **Solution:** Implemented a Soft-Start logic in the moveForward() function to gradually increase the PWM value from 0 to max over a few milliseconds.

Issue 2: Robot Drifting (Not Moving in a Straight Line)

- **Problem:** Even when providing equal PWM values to both motors, the robot drifted to the left, acting like a long left turn.
- **Solution:** This was caused by RPM mismatch between the two DC motors and potential mechanical friction. Software calibration was applied by reducing the RIGHT_MAX speed to 150 while keeping the LEFT_MAX at 190. Additionally, the wheels were physically adjusted to ensure they were not rubbing against the motor chassis.

Issue 3: Incomplete/Over-Rotation during Turns

- **Problem:** The robot was spinning in circles (360 degrees) or taking too long turns when encountering an obstacle.
- **Solution:** 1. Reduced the turning PWM speed to 140. 2. Changed the turning logic from a "Spot Turn" (motors moving in opposite directions) to a "Pivot Turn" (one motor stopped, the other moving forward). This allowed for smoother, more controlled 90-degree turns.

Issue 4: Torque Kick / Jerking After a Turn

- **Problem:** The robot experienced a sudden jerk immediately after completing a turn and attempting to move forward again.
- **Solution:** Added a delay(200); after the stopRobot() function inside the turning functions. This allowed the physical momentum of the robot to settle completely before applying forward power again.